

Sistema Steam Distribuido

Cobertura punto por punto de las rúbricas
Parcial y Final

ICI-4344 — Computación Paralela y Distribuida

Documento de trazabilidad Rúbrica → Código

12 de junio de 2026

Cómo leer este documento. Por cada criterio de las dos rúbricas se responde: **¿Qué se hizo?** (la implementación concreta), **¿Por qué?** (la justificación técnica) y **¿Dónde se ve?** (archivo / clase / método donde está cubierto). La etiqueta **verde** marca lo ya implementado y verificado en código; la **naranja** marca lo que queda como trabajo de informe o presentación.

Índice

1. Resumen del sistema	2
2. Rúbrica PARCIAL — cobertura punto por punto	2
2.1. I. Informe Técnico e Inspección de Modelos (40 %)	2
2.1.1. 1.1 Fundamentación y Teoría — concurrencia, fallos, transparencia	2
2.1.2. 1.2 Modelado de Ingeniería — físico, arquitectónico y de funciones	3
2.1.3. 1.3 Análisis Fundamental — modelo de seguridad y modelo de fallos	3
2.2. II. Implementación y Código Fuente en Java (30 %)	3
2.2.1. 2.1 Comunicación y Marshalling (10 %)	3
2.2.2. 2.2 Gestión de Concurrencia (10 %)	3
2.2.3. 2.3 Lógica de las Funciones Principales (10 %)	4
2.3. III. Presentación, Demo y Defensa (30 %)	4
2.3.1. 3.1 Exposición y Video — 3.2 Defensa Técnica	4
3. Rúbrica FINAL — cobertura punto por punto	4
3.1. §2 Requisitos Técnicos Distribuidos Obligatorios	4
3.1.1. 2.1 Topología multinodo	4
3.1.2. 2.2 Ordenamiento de eventos (Unidad 5) — Relojes de Lamport	4
3.1.3. 2.3 Coordinación distribuida (Unidad 6) — elegir al menos uno	5
3.1.4. 2.4 Tolerancia a fallos	5
3.2. §3 Prueba de Tráfico Real (Carga)	5
3.2.1. 3.1 Generador de carga	5
3.2.2. 3.2 Métricas recolectadas	6
3.2.3. 3.3 Falla inducida durante la prueba	6
3.2.4. 3.4 Evidencia entregada	6
3.3. §4 Pauta de Evaluación	6
3.3.1. 4.1 Teoría — 4.2 Modelado — 4.3 Análisis y Carga (Informe, 35 %)	6
3.3.2. 4.4 Distribución — 4.5 Coordinación y Ordenamiento — 4.6 Tolerancia (Código, 35 %)	7
3.3.3. 4.7 Exposición y Video — 4.8 Demo de Carga y Falla en Vivo — 4.9 Defensa	7
3.4. §5 Entregables	7

4. Apéndice — Puntos a cuidar en la defensa**7**

1. Resumen del sistema

Dominio. Plataforma distribuida tipo *Steam*: catálogo de videojuegos con compra-venta y mensajería entre usuarios. Es la evolución del proyecto parcial (cliente-servidor) a una arquitectura multinodo con coordinación distribuida.

Dos funciones principales de gran escala.

1. **F1 — Compra de juegos (transacción en 2 fases):** reserva temporal con TTL de 5 min sobre *stock finito*, y confirmación de pago contra billetera. Es el recurso crítico protegido por exclusión mutua.
2. **F2 — Mensajería 1-a-1:** envío con almacenamiento *offline* y ordenamiento causal de la conversación mediante relojes de Lamport.

Topología (7 procesos / JVM independientes).

Componente	Rol	Puerto(s)
Proxy	Gateway: ruteo Round-Robin, health-check, membresía	8080
svSesiones 1 / 2	Autenticación y tokens de sesión	8081 / 8181
svJuegos 1 / 2	Transacciones (F1) + Bully + Mutex	8082 / 8282
svMensajería 1/2	Chat (F2) con orden causal	8083 / 8383
(canal Bully)	Elección de coordinador entre nodos de juegos	9082 / 9282
(canal Mutex)	Exclusión mutua del recurso <i>stock</i>	9182 / 9382

Stack. Java (compila con `javac -release 17`), sockets TCP, serialización JSON con Gson 2.10.1. Sin Maven: el script `scripts/1_build.ps1` descarga Gson, compila los 35 archivos y genera `sistema-steam.jar`.

Estructura de paquetes (src/main/java/com/steam/):

- `common/` — Constantes, MensajeProtocolo (sobre JSON), GestorPersistencia, ValidadorToken, Utils, RelojLamport, GestorLog, GestorSnapshot, RegistradorProxy.
- `models/` — Usuario, Sesion, Juego, Reserva, Venta, Mensaje, y las BD en memoria BDSesiones/BDJuegos/BDReservas/BDVentas/BDMensajes.
- `proxy/` — Proxy.
- `servidores/` — svSesiones, svJuegos, svMensajería, GestorLocks.
- `coordinacion/` — GestorBully, GestorMutexCentralizado, RegistroMembresia, NodoInfo, EstadoNodo.
- `carga/` — GeneradorCarga, FallaInducida.
- `watchdog/` — WatchdogServidor.

2. Rúbrica PARCIAL — cobertura punto por punto

2.1. I. Informe Técnico e Inspección de Modelos (40 %)

2.1.1 1.1 Fundamentación y Teoría — concurrencia, fallos, transparencia

Base técnica en código

¿Qué se hizo? Se aplican los tres conceptos clave al sistema. *Concurrencia*: cada servidor atiende muchos clientes a la vez con un pool de hilos y protege los datos compartidos. *Fallos independientes*: cada nodo puede caer por separado sin tumbar al resto. *Transparencia*: el cliente solo conoce el Proxy (puerto 8080) y no sabe qué nodo físico lo atiende.

¿Por qué? Son las tres propiedades intrínsecas de un sistema distribuido (concurrencia, ausencia de reloj global y fallos parciales). El informe debe *vincularlas al código*, no solo definir las.

¿Dónde se ve? Concurrencia: `Executors.newFixedThreadPool` en `svJuegos.escuchar()`. Transparencia de ubicación: `Proxy.rutear()` (el cliente nunca pone IP de un nodo). Transparencia de acceso: protocolo uniforme `MensajeProtocolo` para toda operación.

Diagramas/redacción: informe

2.1.2 1.2 Modelado de Ingeniería — físico, arquitectónico y de funciones

Descrito; faltan diagramas formales

¿Qué se hizo? *Modelo físico*: 7 procesos sobre red local (LAN), comunicados por TCP. *Modelo arquitectónico*: cliente → Proxy (gateway) → clústeres de servidores por servicio (3 capas: cliente, intermediación, servicios + persistencia). *Modelo de funciones*: F1 (compra en 2 fases) y F2 (mensajería) descritas con su paso de mensajes.

¿Por qué? El modelado comunica el diseño antes que el código y permite razonar la arquitectura (es más barato corregir un diagrama que reescribir código).

¿Dónde se ve? La materia prima de los tres modelos está en este documento (Sección 1 y tablas de puertos) y en los encabezados Javadoc de `Proxy.java` y los `sv*.java`. Diagramas UML de secuencia: informe

2.1.3 1.3 Análisis Fundamental — modelo de seguridad y modelo de fallos

Implementado en código

¿Qué se hizo? *Seguridad*: contraseñas con hash SHA-256, tokens de sesión UUID, validación de token en toda operación sensible y control de acceso por rol (COMPRADOR/VENDEDOR/ADMIN). *Fallos*: clasificación Crash (nodo caído) y Omisión (mensaje perdido) con detección por timeouts/health-check y recuperación por failover y respaldo.

¿Por qué? El canal TCP es inseguro (interceptación, suplantación) y los fallos son parciales: hay que autenticar/autorizar y tener estrategia explícita por tipo de fallo.

¿Dónde se ve? Seguridad: `Utils.hashPassword()` (SHA-256), `svSesiones.login()/validarToken()`, `ValidadorToken`, chequeos de rol en `svJuegos.publicarJuego()/agregarSaldo()`. Fallos: `Proxy.iniciarHealth` (Crash) y reintento al espejo en `Proxy.rutear()` (Omisión).

2.2. II. Implementación y Código Fuente en Java (30 %)

2.2.1 2.1 Comunicación y Marshalling (10 %)

Excelente

¿Qué se hizo? Toda la comunicación es por *sockets TCP*. Las estructuras complejas (objetos, listas, mapas anidados) se serializan a JSON con Gson antes de viajar.

¿Por qué? La rúbrica pide sockets + *marshalling de estructuras complejas*, no solo texto plano. El JSON resuelve diferencias de representación entre procesos y permite enviar estados ricos (catálogo, reservas, billeteras) en un solo mensaje.

¿Dónde se ve? `MensajeProtocolo.toJson()/fromJson()` (sobre Gson) y su `Map<String, Object>` payload. Sockets: `ServerSocket/Socket` en `Proxy.escuchar()`, `Proxy.reenviar()` y `sv*.escuchar()`.

Listing 1: Sobre de comunicación serializado a JSON (`com.steam.common.MensajeProtocolo`).

```
1 private static final Gson GSON = new Gson();
2 public String toJson() { return GSON.toJson(this); }
3 public static MensajeProtocolo fromJson(String j){ return GSON.fromJson(j,
    MensajeProtocolo.class); }
```

2.2.2 2.2 Gestión de Concurrencia (10 %)

Excelente

¿Qué se hizo? Servidor multihilo con *pool de hilos* (30 por componente) y sincronización en las regiones críticas: `synchronized` sobre la BD, `Semaphore` para el recurso `stock`, y estructuras *lock-free* (`AtomicLong`, `AtomicInteger`, `ConcurrentHashMap`, `CopyOnWriteArrayList`).

¿Por qué? Un servidor iterativo se bloquea con un cliente lento. El pool da escalabilidad, y la sincronización evita condiciones de carrera (p.ej. que dos compradores reserven el último ejemplar del stock).

¿Dónde se ve? Pool: `Executors.newFixedThreadPool(Constantes.POOL_SIZE)` en cada servidor. Región crítica: bloque `synchronized(lock)` en `svJuegos.comprarJuego()`. Round-Robin atómico: `AtomicInteger` en `Proxy.rutear()`.

2.2.3 2.3 Lógica de las Funciones Principales (10 %)

Excelente

¿Qué se hizo? Las dos funciones operan correctamente y el sistema resiste excepciones de red. F1: `COMPRAR_JUEGO` (reserva + descuento de stock) y `CONFIRMAR_PAGO` (cobro a billetera, registro de venta). F2: `ENVIAR_MENSAJE`, `VER_MENSAJES` (entrega offline), `VER_CONVERSACION`.

¿Por qué? La rúbrica exige dos funciones de gran escala y manejo de excepciones para que un fallo parcial de red no tumbe todo el sistema.

¿Dónde se ve? F1: `svJuegos.comprarJuego()` y `confirmarPago()`; expiración de reservas en `GestorLocks`. F2: `svMensajeria.enviarMensaje()/verMensajes()`. Excepciones: `try/catch` de `IOException` y `SocketTimeoutException` en todos los `manejarCliente()`, con failover en `Proxy.rutear()`.

2.3. III. Presentación, Demo y Defensa (30 %)

2.3.1 3.1 Exposición y Video — 3.2 Defensa Técnica

Pendiente: presentación

¿Qué se hizo? La *capacidad* de demo está lista: se puede levantar el sistema completo y ejercitar ambas funciones en vivo. La coherencia informe↔código está respaldada por este documento.

¿Por qué? La nota de presentación es obligatoria e independiente de la entrega.

¿Dónde se ve? Scripts de ejecución `scripts/2..9`. Material de defensa: este documento y los Javadoc de cada clase.

Grabar video ≤3 min + defensa oral: equipo

3. Rúbrica FINAL — cobertura punto por punto

3.1. §2 Requisitos Técnicos Distribuidos Obligatorios

3.1.1 2.1 Topología multinodo

Excelente

¿Qué se hizo? Tres o más nodos en procesos/JVM separados (7 en total), arquitectura *multiservidor* (clústeres por servicio) donde la coordinación NO está centralizada en un único servidor, y *descubrimiento de membresía*: cada servidor se registra solo en el Proxy al arrancar y se da de baja al apagarse; el estado vivo se persiste.

¿Por qué? El final exige pasar de cliente-servidor a multinodo: ningún nodo solo debe dejar el sistema fuera de servicio, y los nodos deben conocerse por una lista de membresía.

¿Dónde se ve? Registro dinámico: `RegistradorProxy.registrarAsync()` + manejo en `Proxy.procesarRegis`. Membresía persistida: `RegistroMembresia` en `data/MEMBRESIA.txt`. Failover entre nodos del clúster: `Proxy.rutear()`.

3.1.2 2.2 Ordenamiento de eventos (Unidad 5) — Relojes de Lamport

Excelente

¿Qué se hizo? Reloj de Lamport thread-safe propagado en todos los mensajes (cliente → proxy → servidores, y en Bully/Mutex). Al menos una función muestra *orden causal*: la conversación de chat se ordena por marca de Lamport. Cada evento relevante queda en el log con su marca lógica.

¿Por qué? Sin reloj global no hay orden temporal absoluto; Lamport da un orden causal (*happened-before*) suficiente para ordenar la conversación y auditar la coordinación.

¿Dónde se ve? RelojLamport (regla $\max(\text{local}, \text{recibido}) + 1$). Estampado en Proxy.rutear() y svMensajeria.enviarMensaje(); orden causal en svMensajeria.verConversacion() (campo Mensaje.lamportClock). Logs: líneas [LAMPOR] t=, [BULLY] t=, [MUTEX] t=.

Listing 2: Regla de actualización de Lamport en recepción (com.steam.common.RelojLamport).

```

1 public long update(long received) { // evento de recepcion
2     long actual, actualizado;
3     do {
4         actual = reloj.get();
5         actualizado = Math.max(actual, received) + 1;
6     } while (!reloj.compareAndSet(actual, actualizado)); // atomico, sin locks
7     return actualizado;
8 }

```

3.1.3 2.3 Coordinación distribuida (Unidad 6) — elegir al menos uno

Excelente — Bully (elección)

¿Qué se hizo? Se implementó el *algoritmo abusón (Bully)* para elegir coordinador entre los nodos de juegos, que se dispara cuando se detecta la caída del coordinador. Además, ese coordinador administra la *exclusión mutua* del recurso **stock** (mutex con coordinador elegido dinámicamente).

¿Por qué? La rúbrica pide al menos un algoritmo de coordinación; Bully cubre la elección de coordinador “que se dispare solo cuando se detecta que el coordinador cayó”. El mutex sobre el stock evita la sobreventa del último ejemplar entre nodos.

¿Dónde se ve? Elección: GestorBully.iniciarEleccion()/convertirseEnCoordinador() y su daemon de heartbeat. Exclusión mutua: GestorMutexCentralizado.requestLock()/releaseLock() usada en svJuegos.comprarJuego(). *Nota de defensa*: es exclusión mutua *coordinador-céntrica* (no Ricart-Agrawala); el requisito se cumple por la elección Bully.

3.1.4 2.4 Tolerancia a fallos

Excelente

¿Qué se hizo? Detección por *heartbeats y timeouts* en tres niveles y recuperación automática sin caída total: el Proxy hace health-check (10 s) y conmuta al espejo; Bully envía heartbeat al coordinador (5 s) y re-elige si cae; el Watchdog reinicia procesos muertos (15 s, 3 fallos); y un snapshot periódico respalda Main→Copy.

¿Por qué? Cubre tanto Crash (nodo caído) como Omisión (mensaje perdido: timeout + reintento), y garantiza que el servicio sigue tras una caída (re-elección/failover/reinicio).

¿Dónde se ve? Proxy.iniciarHealthCheck(); daemon de heartbeat en GestorBully; WatchdogServidor; respaldo en GestorSnapshot.

3.2. §3 Prueba de Tráfico Real (Carga)

3.2.1 3.1 Generador de carga

Excelente

¿Qué se hizo? Cliente Java que lanza 50 hilos concurrentes durante 60 s sostenidos; cada hilo simula un ciclo de usuario (login → listar → comprar → confirmar → mensaje → logout), ejercitando las dos funciones y, en particular, el recurso protegido por el mutex.

¿Por qué? La rúbrica exige ≥ 50 clientes/hilos por ≥ 60 s y que la carga golpee el recurso en exclusión mutua (el camino de compra pide el lock en cada intento).

¿Dónde se ve? `GeneradorCarga` (parámetros por defecto 50 y 60).

3.2.2 3.2 Métricas recolectadas

Excelente

¿Qué se hizo? Se miden throughput (req/s), latencia promedio y *p95*, cantidad de mensajes del algoritmo de coordinación, y tasa de pérdida. La métrica de error se corrigió para distinguir *pérdida real de transporte* (timeout / sin respuesta) de los rechazos de negocio esperados, de modo que el pico de la falla inducida sea nítido.

¿Por qué? La §3.2 pide exactamente esas métricas, incluida la “cantidad de mensajes que genera el algoritmo de coordinación”.

¿Dónde se ve? Cálculo y reporte en `GeneradorCarga.imprimirReporteFinal()`. Conteo de mensajes de coordinación: contadores `AtomicLong` en `GestorBully` y `GestorMutexCentralizado`, expuestos por la operación `VER_METRICAS_COORD` de `svJuegos` y sumados por `GeneradorCarga.recolectarCoord()`.

Última corrida (50 hilos / 60 s, falla del coordinador @30 s):

Throughput	412 req/s
Latencia p95	219 ms
Peticiones OK	16.045
Pérdida (transporte)	0 %
Mensajes de coordinación	1.964 (Bully = 10, Mutex = 1.954)
Tiempo de recuperación tras la falla	≈ 7 s

3.2.3 3.3 Falla inducida durante la prueba

Excelente

¿Qué se hizo? En plena carga se derriba al coordinador a propósito y se mide cuánto tarda el sistema en reorganizarse. La herramienta localiza al coordinador, le envía un apagado, y mide los milisegundos hasta que el nodo sobreviviente se proclama nuevo coordinador.

¿Por qué? La §3.3 exige derribar al coordinador en vivo y medir recuperación e impacto en latencia/errores.

¿Dónde se ve? `FallaInducida` (consulta `QUIEN_ES_COORDINADOR`, envía `SHUTDOWN_GRACEFUL` y mide la re-elección). El otro nodo re-elige vía `GestorBully`.

3.2.4 3.4 Evidencia entregada

Logs listos; gráfico en informe

¿Qué se hizo? Se conservan los logs de la corrida (marcas lógicas, eventos de elección y detección de la falla) y los datos para la tabla de métricas.

¿Por qué? La §3.4 exige una tabla y al menos un gráfico, más los logs adjuntos.

¿Dónde se ve? Logs: `logs/carga_*.log`, `logs/orq_falla.out`, `logs/svJuegos-*_0.log`, `data/MEMBRESIA.tx`
Orquestador repetible: `scripts/13_run_prueba_trafico.ps1`. Gráfico throughput/latencia vs tiempo: informe

3.3. §4 Pauta de Evaluación

3.3.1 4.1 Teoría — 4.2 Modelado — 4.3 Análisis y Carga (Informe, 35 %)

Pendiente: informe

¿Qué se hizo? La sustancia técnica existe: ausencia de reloj global (Lamport), topología multinodo, modelo de seguridad y de fallos, y la lectura de la prueba de tráfico (Sección §3 de este documento). Falta redactarlo con el formato pregrado y diagramar el UML.

¿Por qué? Es el 35 % del informe; debe interpretar las métricas (throughput, p95, falla inducida), no solo mostrarlas.

¿Dónde se ve? Base para 4.1/4.3 en las secciones §2 y §3 de arriba. **Redacción + diagramas: equipo**

3.3.2 4.4 Distribución — 4.5 Coordinación y Ordenamiento — 4.6 Tolerancia (Código, 35 %)

Excelente

¿Qué se hizo? Estos tres criterios de implementación corresponden directamente a §2.1, §2.2/§2.3 y §2.4 ya descritos: topología real + membresía, Lamport + Bully verificables, y detección/recuperación por heartbeats sin caída total.

¿Por qué? Es el núcleo de código del final y donde se concentra la nota de implementación.

¿Dónde se ve? Ver §2.1 (RegistradorProxy, RegistroMembresía), §2.2 (RelojLamport), §2.3 (GestorBully, GestorMutexCentralizado) y §2.4 (Proxy.iniciarHealthCheck, WatchdogServidor).

3.3.3 4.7 Exposición y Video — 4.8 Demo de Carga y Falla en Vivo — 4.9 Defensa

Pendiente: presentación

¿Qué se hizo? La demo de carga + falla en vivo está *lista para ejecutarse* con un solo comando; solo falta hacerla durante la presentación y grabar el video.

¿Por qué? La §4.8 exige correr la prueba de tráfico en vivo sobre ≥ 3 nodos y derribar un nodo mostrando la recuperación con métricas.

¿Dónde se ve? `scripts/13_run_prueba_trafico.ps1` (levanta los 7 procesos, corre carga + falla y recoge logs/métricas).

Grabar video ≤ 3 min + demo + defensa: equipo

3.4. §5 Entregables

Entregable	Estado	Dónde
Código fuente completo (.zip, incl. generador)	Listo	repositorio + <code>sistema-steam.jar</code>
Logs de carga y falla inducida	Listo	<code>logs/</code>
Informe PDF (formato pregrado + portada)	Pendiente	equipo
Presentación con video demostrativo	Pendiente	equipo

4. Apéndice — Puntos a cuidar en la defensa

- **El Proxy es un punto único de *ruteo***, pero la *coordinación* (Bully, Mutex) está distribuida entre los nodos de juegos, no centralizada. Conviene decirlo así.
- **El mutex es coordinador-céntrico** (no Ricart-Agrawala): preséntelo como “exclusión mutua administrada por el coordinador elegido dinámicamente por Bully”. El requisito §2.3 se cumple por la *elección* Bully.
- **Reloj de Lamport (escalar)**: cumple §2.2. Si preguntan por vectoriales, la respuesta es que Lamport basta para el orden total del chat que se requiere.
- **Recuperación medida (≈ 7 s, 0 % de pérdida)**: el sistema sigue sirviendo durante la caída del coordinador gracias al failover del Proxy al nodo espejo.